

HPC

Arthur Lequertier

September 2, 2026

Contents

1	Introduction	1
2	Les grands principes de l'HPC	3
2.1	La taxonomie de Flynn	3
2.2	Décomposition du problème	4
2.3	La mémoire	4
2.3.1	Les architectures matérielles	4
2.3.2	Les modèles de programmation	6
3	calcul parallèle sur GPU	8
3.1	Qu'est ce qu'un GPU ?	8
3.2	Comment évaluer un GPU	8
3.3	CUDA	9

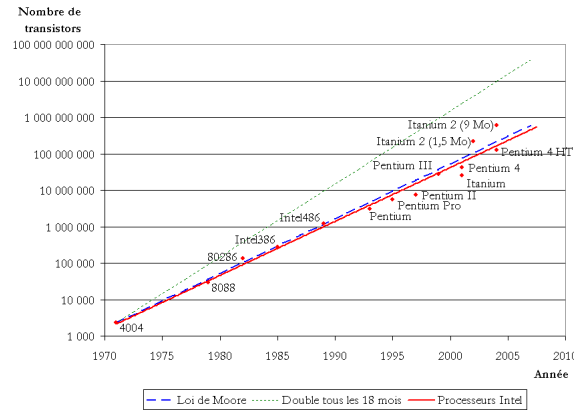
1 Introduction

Le contexte est simple. Les problèmes informatiques sont de plus en plus complexes, de par le nombre de données, la précision exigée, et les interactions à prendre en compte. Heureusement, la technologie de calcul scientifique suit le progrès et permet d'accompagner les nouveaux modèles informatiques ¹ Cependant, un simple processeur de calcul monocœur ne peut pas répondre à lui seul aux attentes des problématiques actuelles. Il est donc nécessaire de concevoir des architectures haute performance, répartissant les tâches entre plusieurs cœurs de calcul.

Pour rappel, la loi de Moore dit que le nombre de transistors intégrés sur une puce double environ tous les 18 à 24 mois, à coût comparable. La miniaturisation permet généralement d'améliorer les performances, la densité et l'efficacité énergétique. Les circuits des processeurs sont gravés de plus en plus finement,

¹Ceci est un raccourci, le progrès sur le software et le hardware va de pair. L'exemple des réseaux de neurones, mis au placard pendant une vingtaine d'années, n'a trouvé son utilité que lorsque les clusters de calcul GPU ont atteint une puissance suffisante pour entraîner de grands modèles.

mais les technologies de fabrication les plus avancées ont atteint une limite, car annoncées autour de 2 nm².



On se rapproche aujourd'hui de la taille d'atome de silicium (environ 0.2 nm) et ne peut évidemment pas continuer indéfiniment à réduire la taille des transistors. Au milieu des années 2000, la fin du Dennard scaling intervint. Jusque-là, réduire la taille des transistors permettait aussi de baisser leur tension, et donc de monter en fréquence, sans exploser la consommation et la dissipation thermique. Mais nous avons atteint une limite, car au-delà d'une certaine finesse de gravure, les courants de fuite deviennent trop importants. Donc, si je devais résumer pourquoi c'est un problème, je répondrais : "Tais-toi, c'est quantique !"

La solution a été d'utiliser ces transistors supplémentaires pour dupliquer des cœurs plutôt que d'accélérer un cœur unique, d'où l'essor du multicœur, et donc la nécessité de paralléliser les logiciels pour en tirer parti.

C'est le principe du calcul parallèle, mais ce n'est pas si simple que ça. Imaginons qu'un programme informatique tournant sur un CPU met 100 s pour tourner. Ce dernier peut être décomposé en un ensemble de tâches. Dans cet exemple, disons que 75 s représente des tâches parallèles (c.-à-d. pouvant s'exécuter simultanément), et 25 s correspondent à une partie qui doit obligatoirement être exécutée séquentiellement. Si on dispose de 10 processeurs, on peut théoriquement diviser les 75 secondes par 10. La partie parallélisable s'exécutera donc en 7.5 s, et la totalité du programme en $7.5 + 25 = 32.5$ s. Le programme est donc $\frac{100}{32.5} \approx 3$ fois plus rapide. La loi d'Amdahl s'écrit généralement

$$S(N) = \frac{1}{(1 - P) + \frac{P}{N}}$$

où :

²Ce sont des tailles caractéristiques utilisées pour différencier les générations de puces, ça ne signifie pas que chaque transistor possède une dimension de 2 nm

- $S(N)$ = accélération (speedup) obtenue avec N processeurs
- P = fraction du programme qui peut être parallélisée
- $1 - P$ = fraction qui doit rester séquentielle
- N = nombre de processeurs

La loi d'Amdahl permet de calculer la limite théorique du speedup lorsque le nombre de processeurs tend vers l'infini. Même une toute petite partie séquentielle peut fortement limiter les performances globales ³.

2 Les grands principes de l'HPC

2.1 La taxonomie de Flynn

Lorsque l'on parle de hardware informatique, il est possible d'utiliser la taxonomie de Flynn pour classer les ordinateurs et les processeurs selon la manière dont ils traitent les instructions et les données en parallèle.

L'idée repose sur deux questions :

1. Combien de flux d'instructions sont exécutés ?
2. Combien de flux de données sont traités ?

On obtient ainsi quatre grandes catégories :

	Un flux de données	Plusieurs flux de données
Une instruction	SISD	SIMD
Plusieurs instructions	MISD	MIMD

- SISD pour Single Instruction, Single Data : c'est le modèle informatique classique, voire élémentaire, où une seule instruction traite une seule donnée à la fois.
- SIMD pour Single Instruction, Multiple Data : ici, une seule instruction est appliquée simultanément à plusieurs données (pour des instructions vectorielles).
- MISD pour Multiple Instruction, Single Data : dans ce modèle, plusieurs instructions différentes travaillent sur un même flux de données.
- MIMD pour Multiple Instruction, Multiple Data : ici, plusieurs unités de calcul peuvent exécuter des instructions différentes sur des données différentes.

Les configurations de type MIMD correspondent le mieux à la plupart des architectures multicœurs modernes, car elles sont adaptées au calcul parallèle et au calcul distribué.

³La loi de Gustafson s'ajoute à la loi d'Amdahl, car en pratique, quand on a plus de ressources, on a tendance à augmenter la taille du problème plutôt qu'à garder la taille fixe.

2.2 Décomposition du problème

Lorsque l'on aborde un problème computationnel, deux grandes approches complémentaires s'offrent à nous.

- **Décomposition de données** : on découpe le domaine (ex. un maillage CFD, un réseau de neurones) en sous-domaines traités en parallèle.
- **Décomposition de tâches** : on découpe le problème en tâches (threads) distinctes sur le plan fonctionnel.

2.3 La mémoire

La gestion de la mémoire est l'un des aspects les plus importants du calcul parallèle. En effet, disposer de nombreux cœurs de calcul ne suffit pas, car ces cœurs doivent aussi pouvoir accéder aux données dont ils ont besoin, et cet accès peut devenir le principal facteur limitant des performances. En effet, l'ordonnancement des tâches est important, car des résultats de threads peuvent être utiles à d'autres, les rendant limitants (on peut voir ça comme un diagramme de Gantt. On souhaite répartir le travail entre plusieurs unités de calcul).

Si on souhaite répartir le travail d'un programme entre plusieurs unités de calcul, une question essentielle se pose ! Où sont stockées les données ? Les cœurs de calcul partagent-ils la même mémoire ou bien chaque cœur possède-t-il sa propre mémoire ?

La réponse à cette question conditionne fortement le modèle de programmation parallèle utilisé. Il convient cependant de distinguer deux notions bien différentes :

- **l'architecture matérielle** : où sont physiquement stockées les données, et qui peut y accéder directement.
- **le modèle de programmation** : la façon dont le code est écrit pour exploiter cette architecture.

Chaque problème computationnel présente ses avantages et ses inconvénients en matière de parallélisation. Il est ainsi nécessaire de connaître les différentes architectures de mémoire distribuée, ainsi que les modèles de programmation utilisés pour manipuler cette mémoire.

2.3.1 Les architectures matérielles

La mémoire partagée Dans une architecture à mémoire partagée, plusieurs processeurs ou cœurs peuvent accéder à un espace mémoire commun. Ainsi, tous les processeurs peuvent potentiellement accéder aux mêmes variables.

NUMA (Non-Uniform Memory Access) Concrètement : sur une machine multi-nœuds (plusieurs puces CPU sur la même carte mère, par exemple), chaque nœud (ou socket) dispose de RAM directement rattachée localement. Un cœur peut aussi accéder à la RAM rattachée à un autre nœud, mais il doit alors passer par un lien d'interconnexion entre les deux puces (Intel UPI, AMD Infinity Fabric), ce qui ajoute de la latence et réduit la bande passante disponible. Ainsi, deux threads sur le même nœud accédant à la même RAM locale devraient, a priori, bénéficier d'un accès rapide et d'une latence uniforme. Cependant, un thread sur le nœud 1, qui accède en boucle à une zone mémoire allouée à proximité d'un autre nœud, entraîne un ralentissement systématique.

NUMA désigne les systèmes multiprocesseurs à mi-chemin entre la mémoire partagée et la mémoire distribuée, dans lesquels les zones mémoire sont séparées et placées dans différents emplacements. Du point de vue de l'architecture, NUMA reste une forme de mémoire partagée : l'espace d'adressage est unique, et n'importe quel cœur peut en théorie accéder à n'importe quelle donnée. Mais contrairement à une architecture UMA (*Uniform Memory Access*) où ce temps d'accès est le même pour tous, NUMA introduit une asymétrie : les temps d'accès diffèrent selon la zone mémoire accédée. C'est en ce sens que NUMA se situe à mi-chemin entre mémoire partagée et mémoire distribuée : l'architecture reste partagée, mais son comportement en pratique (notamment en termes de performance) se rapproche par endroits de celui d'une mémoire distribuée. On appelle ainsi *Facteur NUMA* le rapport entre le temps d'accès à une mémoire distante et à une mémoire locale.

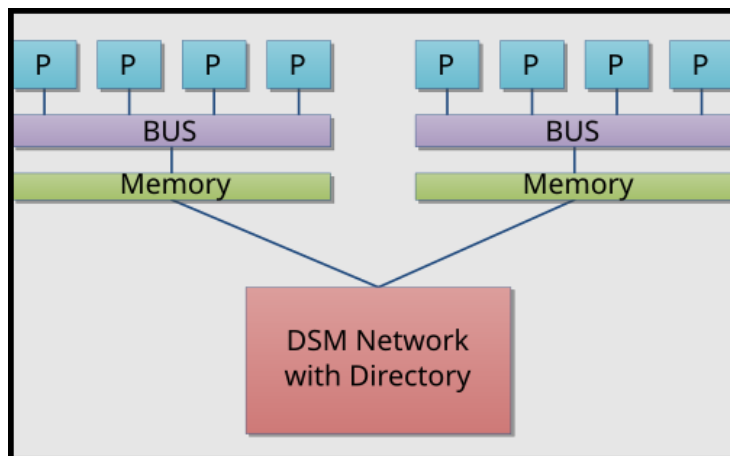


Figure 1: Une architecture possible d'un système NUMA. Les processeurs sont reliés au bus par un nombre variable de connexions. Cela montre que les différents processeurs ont des priorités d'accès à la mémoire qui varient selon leur position relative.

Le piège est que selon les outils utilisés, le programmeur pourrait voir un seul espace d'adressage unifié (comme en mémoire partagée), mais les performances

se comportent en partie comme en mémoire distribuée, car la localité compte. L'API classique `malloc/threads` masque la topologie NUMA, alors que des outils comme `numactl/libnuma` l'exposent.

La mémoire distribuée Dans une architecture à mémoire distribuée, chaque processeur ou chaque nœud de calcul possède sa propre mémoire locale. Chaque processeur doit alors échanger explicitement des données avec les autres.

Les architectures hybrides Les architectures modernes combinent souvent les deux approches. Un supercalculateur peut être constitué de nombreux nœuds, chacun possédant plusieurs cœurs.

2.3.2 Les modèles de programmation

Une architecture donnée n'impose pas un unique modèle de programmation : certaines associations sont simplement les plus naturelles, car les plus efficaces. C'est ce qu'on va voir dans les sous-sections suivantes.

OpenMP Le modèle de programmation associé à la mémoire partagée est le plus souvent **OpenMP**, une API permettant de programmer ce type d'architecture. Le principe est de créer plusieurs threads qui travaillent en parallèle. Par exemple, si on travaille sur une boucle, il est possible de répartir cette dernière entre plusieurs threads.

```
1 #pragma omp parallel for
2 for (int i = 0; i < N; i++) {
3     C[i] = A[i] + B[i];
4 }
```

C'est très pratique pour les problèmes où les itérations sont relativement indépendantes.

La mémoire partagée pose cependant une difficulté majeure, car plusieurs threads peuvent modifier les mêmes données simultanément. C'est ce qu'on appelle une **condition de course** (race condition). Il faut donc utiliser des mécanismes de synchronisation (i.e., mutex, locks, barrières, etc...).

Une autre limite de la mémoire partagée est que la mémoire et les communications deviennent un goulot d'étranglement lorsque le nombre de processeurs augmente (problème de bande passante et de cohérence des caches). C'est l'une des raisons pour lesquelles les supercalculateurs recourent souvent à un modèle de mémoire distribuée.

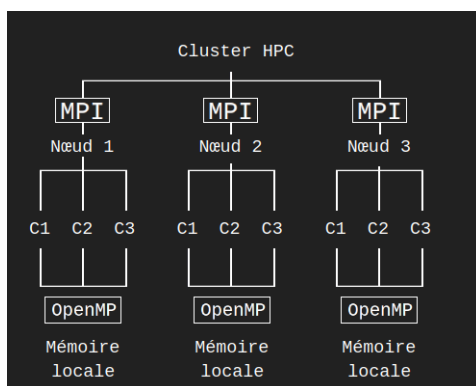
MPI Le modèle de programmation associé à la mémoire distribuée est **MPI** (Message Passing Interface), un standard qui permet à plusieurs processus de

communiquer en échangeant des messages ⁴. Son principal avantage est la **scalabilité**. C'est-à-dire que si on veut augmenter la puissance de calcul, on peut ajouter des nœuds. Car chaque nouveau nœud apporte :

- Des processeurs supplémentaires
- De la mémoire supplémentaire
- Une capacité de calcul supplémentaire

L'approche hybride Par exemple, supposons que l'on souhaite traiter un ensemble de données énorme. MPI distribue les données entre les nœuds et OpenMP répartit ensuite le travail entre les cœurs de chaque nœud. On obtient ainsi deux niveaux de parallélisme.

- MPI entre les nœuds
- OpenMP à l'intérieur de chaque nœud



Par exemple, supposons que l'on souhaite traiter un ensemble de données énorme. MPI distribue les données entre les nœuds et OpenMP répartit ensuite le travail entre les cœurs de chaque nœud. On obtient ainsi deux niveaux de parallélisme.

Cependant, dans les architectures parallèles modernes, le coût des déplacements de données n'est pas à négliger par rapport au coût de calcul pur, bien au contraire. On peut avoir un processeur extrêmement puissant, mais si celui-ci doit attendre les données provenant de la mémoire ou d'un autre nœud, ses unités de calcul restent inactives ; c'est la hiérarchie mémoire (registre → cache L1 → cache L2 → cache L3 → RAM → mémoire d'un autre nœud sur le réseau). La performance d'une application parallèle dépend donc fortement de la manière dont les données circulent au sein de cette hiérarchie.

⁴Bien que conçu pour la mémoire distribuée, MPI n'est pas incompatible avec la mémoire partagée : plusieurs processus MPI sur un même nœud peuvent très bien communiquer en utilisant la mémoire partagée comme support de transport, sans passer par le réseau. Le modèle de programmation (passage de messages explicite) ne dépend donc pas strictement de l'architecture sous-jacente.

3 calcul parallèle sur GPU

3.1 Qu'est ce qu'un GPU ?

Pour Graphics Processing Unit, il s'agit d'unités de calcul dédiées au calcul, initialement conçues pour le rendu graphique. Alors qu'un CPU moderne possède généralement un nombre relativement limité de cœurs très puissants, un GPU possède énormément d'unités de calcul. Ainsi, un CPU fera rapidement des tâches complexes et variées, et un GPU sera spécialisé dans la réalisation de la même opération en parallèle énormément de fois.

3.2 Comment évaluer un GPU

Un GPU présente plusieurs caractéristiques dépendant de sa conception.

- La fréquence : correspond à la vitesse à laquelle son processeur effectue des calculs, mesurée en mégahertz (MHz).
- Nombre de cœurs de calcul : La puce d'une GPU est constituée d'un certain nombre de cœurs de calcul (cœurs CUDA chez NVIDIA, et cœurs de flux chez AMD). Chacun de ces cœurs est une unité de calcul exploitable pour paralléliser les calculs. Cependant, les cœurs de calcul de marques différentes et même de GPU de génération différente ne sont pas forcément comparables, ce qui fait de ce nombre un indicateur approximatif de la performance d'un GPU avant même de l'utiliser.
- Le TFLOPS : Cette unité mesure la puissance de calcul d'un processeur ou d'une carte graphique. T pour tera (10^{12}), FLOP pour *float operation*, et le S pour s^{-1} , il correspond à mille milliards d'opérations en virgule flottante par seconde, c'est-à-dire des calculs mathématiques avec des nombres décimaux. Plus ce nombre est élevé, plus l'appareil est rapide pour gérer des calculs.
- La VRAM : Il s'agit de la mémoire vive dédiée à la carte graphique. Bien que n'intervenant pas dans la vitesse des calculs, une VRAM trop petite peut être un goulot d'étranglement si le nombre de variables du calcul parallèle devient trop élevé.

Tous les paramètres ci-dessus jouent un rôle clé dans l'évaluation des performances d'un GPU ; cependant, le facteur économique et environnemental est crucial dans le choix de la carte. Ainsi, le rapport TFLOPS/prix est le premier facteur pris en compte dans le choix d'une unité de calcul GPU. De plus, les GPU consomment énormément de puissance électrique pour fonctionner (de 100 à 500 watts) et croissent davantage au fil des générations. Une grande partie de la puissance introduite se trouve dissipée sous forme de chaleur. Mais, pour bien fonctionner, les centres de calcul doivent maintenir la température des GPU à

une température convenable (entre 50 et 80 °C), alors que leur fonctionnement est exothermique⁵.

3.3 CUDA

CUDA signifie "Compute Unified Device Architecture". C'est une plateforme de calcul parallèle développée par NVIDIA qui permet d'utiliser les GPU NVIDIA pour des calculs généraux, et pas seulement pour le rendu graphique.

CUDA utilise une organisation appelée SIMT (Single Instruction, Multiple Threads), une extension de SIMD dans la taxonomie de Flynn. Un thread est une instruction de calcul à exécuter. Une partie des threads est regroupée en blocs. Les threads d'un même bloc peuvent collaborer et partager certaines données via la mémoire partagée du GPU. Les blocs sont regroupés dans le grid qui constitue l'ensemble du noyau (kernel) CUDA. Tous les threads exécutent généralement le même noyau, mais chaque thread travaille sur des données différentes. Pour l'addition de deux vecteurs, cela prend la forme suivante

```
1 __global__ void addition(float *A, float *B, float *C)
2 {
3     int i = threadIdx.x;
4     C[i] = A[i] + B[i];
5 }
```

⁵De plus en plus de supercalculateurs (tel Jean Zay) utilisent un système de refroidissement par eau chaude à cœur qui récupère une grande partie des calories émises pour chauffer les réseaux d'urbanisme.